

Bhuddewar Arun

2303A51735

```
import java.util.*;

public class tasksegmentGCD {

    static int tree[];

    static void build(int ind,int st,int ed,int arr[]){

        if(st==ed){

            tree[ind]=arr[st];

            return;

        }

        int mid=(st+ed)/2;

        build(2*ind+1,st,mid,arr);

        build(2*ind+2,mid+1,ed,arr);

        tree[ind]=findGCD(tree[2*ind+1],tree[2*ind+2]);

    }

    static int findGCD(int a,int b){

        return (b==0)?a:findGCD(b,a%b);

    }

    static int find(int ind,int st,int ed,int arr[],int i,int j){

        if(i>ed || j<st) return 0;

        if(i<=st && ed<=j){

            return tree[ind];

        }

        int mid=(st+ed)/2;
```

```

    int left=find(2*ind+1,st,mid,arr,i,j);

    int right=find(2*ind+2,mid+1,ed,arr,i,j);

    return findGCD(left, right);
}

static void update(int ind,int st,int ed,int arr[],int index,int val){

    if(st==ed){

        tree[ind]=val;

        return ;

    }

    int mid=(st+ed)/2;

    if(index<=mid){

        update(ind,st,mid,arr,index,val);

    }else{

        update(ind,mid+1,ed,arr,index,val);

    }

    tree[ind]=findGCD(tree[2*ind+1],tree[2*ind+2]);

}

public static void main(String []args){

    Scanner obj=new Scanner(System.in);

    int n=obj.nextInt();

    int arr[]=new int[n];

    tree=new int[4*n];

    for(int i=0;i<n;i++){

        arr[i]=obj.nextInt();

    }
}

```

```

int q=obj.nextInt();
build(0,0,n-1,arr);
int res[]=new int[q];
int i=0;
while(q-->0){
    int type=obj.nextInt();
    if(type==1){
        int index=obj.nextInt();
        int val=obj.nextInt();
        arr[index]=val;
        update(0,0,n-1,arr,index,val);
    }else if(type==2){
        int a=obj.nextInt();
        int b=obj.nextInt();
        System.out.println(find(0,0,n-1,arr,a,b));
    }
}

for(int m:res){
    System.out.println(m);
}
}

```

2 .

```

import java.util.*;

public class segmentproduct {

```

```

static int tree[];

static int mod=1000000007;

static void build(int ind,int st,int ed,int arr[]){
    if(st==ed){
        tree[ind]=arr[st];
        return ;
    }

    int mid=(st+ed)/2;
    build(2*ind+1,st,mid,arr);
    build(2*ind+2,mid+1,ed,arr);

    tree[ind]=(tree[2*ind+1]*tree[2*ind+2])%mod;
}

public static void update(int ind,int st,int ed,int arr[],int index,int val){

    if(st==ed){
        tree[ind]=val;
        return ;
    }

    int mid=(st+ed)/2;
    if(index<=mid){
        update(2*ind+1,st,mid,arr,index,val);
    }else{
        update(2*ind+2,mid+1,ed,arr,index,val);
    }

    tree[ind] = (int)((long) tree[2 * ind + 1]
        * tree[2 * ind + 2] % mod);
}

```

```

}

public static int rangePro(int ind,int st,int ed,int arr[],int i,int j){

    if(i>ed || j<st) return 1;

    if(st<=i && ed<=j){

        return tree[ind];

    }

    int mid=(st+ed)/2;

    int left=rangePro(2*ind+1,st,mid,arr,i,j);

    int right=rangePro(2*ind+2,mid+1,ed,arr,i,j);

    return tree[ind]=(left*right)%mod;

}

public static void main(String args[]){

    Scanner obj=new Scanner(System.in);

    int n=obj.nextInt();

    int arr[]=new int[n];

    tree=new int[4*n];

    for(int i=0;i<n;i++){

        arr[i]=obj.nextInt();

    }

    int q=obj.nextInt();

    build(0,0,n-1,arr);

    while(q-->0){

        int type=obj.nextInt();

        if(type==1){

```

```

        int index=obj.nextInt();

        int val=obj.nextInt();

        arr[index]=val;

        update(0,0,n-1,arr,index,val);
    }else if(type==2){

        int a=obj.nextInt();

        int b=obj.nextInt();

        System.out.println(rangePro(0,0,n-1,arr,a,b));

    }

}

}

```

3.

```

import java.util.*;

public class segmentbitwise {

    static int tree[];

    static void build(int ind,int st,int ed,int arr[]){

        if(st==ed){

            tree[ind]=arr[st];

            return ;

        }

        int mid=(st+ed)/2;

        build(2*ind+1,st,mid,arr);

        build(2*ind+2,mid+1,ed,arr);
    }
}

```

```

    tree[ind]=(tree[2*ind+1] | tree[2*ind+2]);
}

public static void update(int ind,int st,int ed,int arr[],int index,int val){

    if(st==ed){
        tree[ind]=val;
        return ;
    }
    int mid=(st+ed)/2;
    if(index<=mid){
        update(2*ind+1,st,mid,arr,index,val);
    }else{
        update(2*ind+2,mid+1,ed,arr,index,val);
    }
    tree[ind] = ( tree[2 * ind + 1]
        | tree[2 * ind + 2]);

}

public static int rangeBit(int ind,int st,int ed,int arr[],int i,int j){
    if(i>ed || j<st) return 0;
    if(i<=st && ed<=j){
        return tree[ind];
    }
    int mid=(st+ed)/2;
    int left=rangeBit(2*ind+1,st,mid,arr,i,j);
    int right=rangeBit(2*ind+2,mid+1,ed,arr,i,j);
    return (left | right);
}

```

```

}

public static void main(String args[]){

    Scanner obj=new Scanner(System.in);

    int n=obj.nextInt();

    int arr[]=new int[n];

    tree=new int[4*n];

    for(int i=0;i<n;i++){

        arr[i]=obj.nextInt();

    }

    int q=obj.nextInt();

    build(0,0,n-1,arr);

    while(q-->0){

        int type=obj.nextInt();

        if(type==1){

            int index=obj.nextInt();

            int val=obj.nextInt();

            arr[index]=val;

            update(0,0,n-1,arr,index,val);

        }else if(type==2){

            int a=obj.nextInt();

            int b=obj.nextInt();

            System.out.println(rangeBit(0,0,n-1,arr,a,b));

        }

    }

}

```



```
}
```

4.

```
import java.util.*;
```

```
public class kthone {
```

```
    static int tree[];
```

```
    // Build Segment Tree
```

```
    // Each node stores the number of 1s in that segment
```

```
    static void build(int ind, int st, int ed, int arr[]) {
```

```
        // Leaf node
```

```
        if (st == ed) {
```

```
            tree[ind] = arr[st];
```

```
            return;
```

```
        }
```

```
        int mid = (st + ed) / 2;
```

```
        // Build left child
```

```
        build(2 * ind + 1, st, mid, arr);
```

```
        // Build right child
```

```
        build(2 * ind + 2, mid + 1, ed, arr);
```

```
// Number of 1s = left count + right count
tree[ind] =
    tree[2 * ind + 1] +
    tree[2 * ind + 2];
}
```

```
// Point Update
```

```
static void update(
```

```
    int ind,
```

```
    int st,
```

```
    int ed,
```

```
    int index,
```

```
    int value) {
```

```
// Reached the required position
```

```
if (st == ed) {
```

```
    tree[ind] = value;
```

```
    return;
```

```
}
```

```
int mid = (st + ed) / 2;
```

```
// Go to left child
```

```
if (index <= mid) {
```

```
    update(
```

```
        2 * ind + 1,
```

```
        st,
```

```

        mid,
        index,
        value
    );

} else {

    // Go to right child
    update(
        2 * ind + 2,
        mid + 1,
        ed,
        index,
        value
    );
}

// Recalculate parent
tree[ind] =
    tree[2 * ind + 1] +
    tree[2 * ind + 2];
}

// Find index of k-th one
static int findKthOne(
    int ind,
    int st,
    int ed,

```

```

int k) {

// Reached a leaf
if (st == ed) {
    return st;
}

int mid = (st + ed) / 2;

// Number of ones in left child
int leftCount = tree[2 * ind + 1];

/*
 * If left child contains at least k ones,
 * the k-th one is somewhere in the left child.
 */
if (k <= leftCount) {

    return findKthOne(
        2 * ind + 1,
        st,
        mid,
        k
    );

} else {

/*

```

```

    * The k-th one is in the right child.
    *
    * We skip all the ones in the left child,
    * so reduce k by leftCount.
    */
    return findKthOne(
        2 * ind + 2,
        mid + 1,
        ed,
        k - leftCount
    );
}
}

```

```

public static void main(String[] args) {

    Scanner obj = new Scanner(System.in);

    int n = obj.nextInt();

    int arr[] = new int[n];

    tree = new int[4 * n];

    // Read binary array
    for (int i = 0; i < n; i++) {
        arr[i] = obj.nextInt();
    }
}

```

```
// Build Segment Tree
build(0, 0, n - 1, arr);

int q = obj.nextInt();

while (q-- > 0) {

    int type = obj.nextInt();

    // Type 1: Point Update
    // 1 index value
    if (type == 1) {

        int index = obj.nextInt();
        int value = obj.nextInt();

        arr[index] = value;

        update(
            0,
            0,
            n - 1,
            index,
            value
        );
    }
}
```

```

// Type 2: Find k-th one
// 2 k
else if (type == 2) {

    int k = obj.nextInt();

    // Total number of ones
    int totalOnes = tree[0];

    // Not enough ones
    if (totalOnes < k) {
        System.out.println(-1);
    } else {

        int answer = findKthOne(
            0,
            0,
            n - 1,
            k
        );

        System.out.println(answer);
    }
}
}
}
}
}

```

5.

```
import java.util.*;
```

```
public class segmentfirst {
```

```
    static int tree[];
```

```
    static void build(int ind, int st, int ed, int arr[]) {
```

```
        if (st == ed) {
```

```
            tree[ind] = arr[st];
```

```
            return;
```

```
        }
```

```
        int mid = (st + ed) / 2;
```

```
        build(2 * ind + 1, st, mid, arr);
```

```
        build(2 * ind + 2, mid + 1, ed, arr);
```

```
        tree[ind] = Math.max(tree[2 * ind + 1], tree[2 * ind + 2]);
```

```
    }
```

```
    static void update(int ind, int st, int ed, int index, int val) {
```

```
        if (st == ed) {
```

```
            tree[ind] = val;
```

```
            return;
```

```
        }
```



```

int mid = (st + ed) / 2;

if (index <= mid)
    update(2 * ind + 1, st, mid, index, val);
else
    update(2 * ind + 2, mid + 1, ed, index, val);

tree[ind] = Math.max(tree[2 * ind + 1], tree[2 * ind + 2]);
}

static int find(int ind, int st, int ed, int l, int r, int x) {
    if (ed < l || st > r || tree[ind] < x)
        return -1;

    if (st == ed)
        return st;

    int mid = (st + ed) / 2;

    int left = find(2 * ind + 1, st, mid, l, r, x);

    if (left != -1)
        return left;

    return find(2 * ind + 2, mid + 1, ed, l, r, x);
}

public static void main(String args[]) {

```

```
Scanner obj = new Scanner(System.in);
```

```
int n = obj.nextInt();
```

```
int arr[] = new int[n];
```

```
tree = new int[4 * n];
```

```
for (int i = 0; i < n; i++)
```

```
    arr[i] = obj.nextInt();
```

```
build(0, 0, n - 1, arr);
```

```
int q = obj.nextInt();
```

```
while (q-- > 0) {
```

```
    int type = obj.nextInt();
```

```
    if (type == 1) {
```

```
        int index = obj.nextInt();
```

```
        int val = obj.nextInt();
```

```
        arr[index] = val;
```

```
        update(0, 0, n - 1, index, val);
```

```
    } else {
```

```
        int l = obj.nextInt();
```

```
        int r = obj.nextInt();
```

```
        int x = obj.nextInt();
```

```

        System.out.println(find(0, 0, n - 1, l, r, x));
    }
}
}
}

```

6.

```

import java.util.*;

public class segmentzero {

    static int tree[];

    static void build(int ind, int st, int ed, int arr[]) {
        if (st == ed) {
            tree[ind] = arr[st] == 0 ? 1 : 0;
            return;
        }

        int mid = (st + ed) / 2;

        build(2 * ind + 1, st, mid, arr);
        build(2 * ind + 2, mid + 1, ed, arr);

        tree[ind] = tree[2 * ind + 1] + tree[2 * ind + 2];
    }
}

```

```

static void update(int ind, int st, int ed, int index, int val) {
    if (st == ed) {
        tree[ind] = val == 0 ? 1 : 0;
        return;
    }

    int mid = (st + ed) / 2;

    if (index <= mid)
        update(2 * ind + 1, st, mid, index, val);
    else
        update(2 * ind + 2, mid + 1, ed, index, val);

    tree[ind] = tree[2 * ind + 1] + tree[2 * ind + 2];
}

```

```

static int find(int ind, int st, int ed, int l, int r) {
    if (l > ed || r < st)
        return 0;

    if (l <= st && ed <= r)
        return tree[ind];

    int mid = (st + ed) / 2;

    int left = find(2 * ind + 1, st, mid, l, r);
    int right = find(2 * ind + 2, mid + 1, ed, l, r);
}

```

```

        return left + right;
    }

    public static void main(String args[]) {
        Scanner obj = new Scanner(System.in);

        int n = obj.nextInt();
        int arr[] = new int[n];
        tree = new int[4 * n];

        for (int i = 0; i < n; i++)
            arr[i] = obj.nextInt();

        build(0, 0, n - 1, arr);

        int q = obj.nextInt();

        while (q-- > 0) {
            int type = obj.nextInt();

            if (type == 1) {
                int index = obj.nextInt();
                int val = obj.nextInt();

                arr[index] = val;
                update(0, 0, n - 1, index, val);

            } else {

```

```

        int l = obj.nextInt();

        int r = obj.nextInt();

        System.out.println(find(0, 0, n - 1, l, r));
    }
}
}
}

```

7.

```

import java.util.*;

public class segmentlongestone {

    static class Node {
        int prefix, suffix, best, len;

        Node(int prefix, int suffix, int best, int len) {
            this.prefix = prefix;
            this.suffix = suffix;
            this.best = best;
            this.len = len;
        }
    }

    static Node tree[];

```

```

static Node merge(Node a, Node b) {
    if (a == null)
        return b;

    if (b == null)
        return a;

    int prefix = a.prefix;
    int suffix = b.suffix;

    if (a.prefix == a.len)
        prefix = a.len + b.prefix;

    if (b.suffix == b.len)
        suffix = b.len + a.suffix;

    int best = Math.max(a.best, b.best);
    best = Math.max(best, a.suffix + b.prefix);

    return new Node(prefix, suffix, best, a.len + b.len);
}

```

```

static void build(int ind, int st, int ed, int arr[]) {
    if (st == ed) {
        tree[ind] = new Node(arr[st], arr[st], arr[st], 1);
        return;
    }
}

```

```

int mid = (st + ed) / 2;

build(2 * ind + 1, st, mid, arr);
build(2 * ind + 2, mid + 1, ed, arr);

tree[ind] = merge(tree[2 * ind + 1], tree[2 * ind + 2]);
}

static void update(int ind, int st, int ed, int index, int val) {
    if (st == ed) {
        tree[ind] = new Node(val, val, val, 1);
        return;
    }

    int mid = (st + ed) / 2;

    if (index <= mid)
        update(2 * ind + 1, st, mid, index, val);
    else
        update(2 * ind + 2, mid + 1, ed, index, val);

    tree[ind] = merge(tree[2 * ind + 1], tree[2 * ind + 2]);
}

static Node find(int ind, int st, int ed, int l, int r) {
    if (l > ed || r < st)
        return null;

```



```

    if (l <= st && ed <= r)
        return tree[ind];

    int mid = (st + ed) / 2;

    Node left = find(2 * ind + 1, st, mid, l, r);
    Node right = find(2 * ind + 2, mid + 1, ed, l, r);

    return merge(left, right);
}

public static void main(String args[]) {
    Scanner obj = new Scanner(System.in);

    int n = obj.nextInt();
    int arr[] = new int[n];
    tree = new Node[4 * n];

    for (int i = 0; i < n; i++)
        arr[i] = obj.nextInt();

    build(0, 0, n - 1, arr);

    int q = obj.nextInt();

    while (q-- > 0) {
        int type = obj.nextInt();

```

```

    if (type == 1) {
        int index = obj.nextInt();
        int val = obj.nextInt();

        arr[index] = val;
        update(0, 0, n - 1, index, val);

    } else {
        int l = obj.nextInt();
        int r = obj.nextInt();

        System.out.println(find(0, 0, n - 1, l, r).best);
    }
}
}
}
}

```

8.

```

import java.util.*;

public class segmentpairsum {

    static class Node {
        long max1, max2;

        Node(long max1, long max2) {

```

```
        this.max1 = max1;

        this.max2 = max2;
    }
}
```

```
static Node tree[];
```

```
static Node merge(Node a, Node b) {
    long max1 = Math.max(a.max1, b.max1);

    long max2 = Math.max(
        Math.min(a.max1, b.max1),
        Math.max(a.max2, b.max2)
    );

    return new Node(max1, max2);
}
```

```
static void build(int ind, int st, int ed, int arr[]) {
    if (st == ed) {
        tree[ind] = new Node(arr[st], Long.MIN_VALUE);
        return;
    }
```

```
    int mid = (st + ed) / 2;
```

```
    build(2 * ind + 1, st, mid, arr);
```

```
    build(2 * ind + 2, mid + 1, ed, arr);
```

```
    tree[ind] = merge(tree[2 * ind + 1], tree[2 * ind + 2]);  
}
```

```
static void update(int ind, int st, int ed, int index, int val) {  
    if (st == ed) {  
        tree[ind] = new Node(val, Long.MIN_VALUE);  
        return;  
    }
```

```
    int mid = (st + ed) / 2;
```

```
    if (index <= mid)  
        update(2 * ind + 1, st, mid, index, val);  
    else  
        update(2 * ind + 2, mid + 1, ed, index, val);
```

```
    tree[ind] = merge(tree[2 * ind + 1], tree[2 * ind + 2]);  
}
```

```
static Node find(int ind, int st, int ed, int l, int r) {  
    if (l > ed || r < st)  
        return null;  
  
    if (l <= st && ed <= r)  
        return tree[ind];  
  
    int mid = (st + ed) / 2;
```

```

Node left = find(2 * ind + 1, st, mid, l, r);
Node right = find(2 * ind + 2, mid + 1, ed, l, r);

if (left == null)
    return right;

if (right == null)
    return left;

return merge(left, right);
}

```

```

public static void main(String args[]) {
    Scanner obj = new Scanner(System.in);

    int n = obj.nextInt();
    int arr[] = new int[n];
    tree = new Node[4 * n];

    for (int i = 0; i < n; i++)
        arr[i] = obj.nextInt();

    build(0, 0, n - 1, arr);

    int q = obj.nextInt();

    while (q-- > 0) {

```

```

int type = obj.nextInt();

if (type == 1) {
    int index = obj.nextInt();
    int val = obj.nextInt();

    arr[index] = val;
    update(0, 0, n - 1, index, val);

} else {
    int l = obj.nextInt();
    int r = obj.nextInt();

    Node ans = find(0, 0, n - 1, l, r);

    System.out.println(ans.max1 + ans.max2);
}
}
}
}

```

9.

```

import java.util.*;

public class segmentfrequency {

    static class Node {
        int leftVal, rightVal;

```

```
int prefix, suffix, best;
```

```
Node(int leftVal, int rightVal, int prefix, int suffix, int best) {
```

```
    this.leftVal = leftVal;
```

```
    this.rightVal = rightVal;
```

```
    this.prefix = prefix;
```

```
    this.suffix = suffix;
```

```
    this.best = best;
```

```
}
```

```
}
```

```
static Node tree[];
```

```
static Node
```